

LATIHAN

Analisis Kebutuhan

Berdasarkan soal, diperlukan tiga struktur linked list berbeda dari Chapter 9 yang bekerja secara bersamaan dalam satu aplikasi:

Fitur	Struktur Data	Referensi Materi
Multiple tags per note	Multi-Linked List	Section 9.3 — Gambar 9.10
Chronological view	Sorted DLL (by date)	Section 9.1 — Listing 9.4
Alphabetical view	Sorted DLL (by title)	Section 9.1 — Listing 9.2
Sync status tracking	Circular Linked List	Section 9.2 — Listing 9.5

Implementasi Struktur Data

Kode 1 — NoteNode: Multi-Linked Node Structure

Setiap note memiliki dua pasang pointer doubly linked (untuk chain chronological dan alphabetical) serta dictionary `next_by_tag` untuk chain per-tag. Konsep ini mengacu pada multi-linked list Section 9.3 di mana satu node bisa masuk ke banyak chain sekaligus.

```
1. NoteNode - Multi-Linked Node Structure

1 class NoteNode:
2     def __init__(self, title, content, date, tags):
3         self.title = title      # judul note
4         self.content = content  # isi note
5         self.date = date       # tanggal (format: 'YYYY-MM-DD')
6         self.tags = tags       # list of tag strings
7
8         # Doubly linked - chronological chain (Section 9.1)
9         self.next_chron = None
10        self.prev_chron = None
11
12        # Doubly linked - alphabetical chain (Section 9.1)
13        self.next_alpha = None
14        self.prev_alpha = None
15
16        # Multi-linked - tag chains (Section 9.3)
17        # key: tag string, value: NoteNode berikutnya dengan tag sama
18        self.next_by_tag = {}
```

Kode 2 — Insert Chronological: Sorted Doubly Linked List (by Date)

Menggunakan 4 kasus insert sesuai Listing 9.4 buku: (1) list kosong, (2) insert di depan, (3) insert di belakang, (4) insert di tengah. Setiap insert juga mengatur pointer `prev` agar bisa traversal bolak-balik.

```
2. Insert Chronological - Sorted Doubly Linked List

1 def _insert_chron(self, new_node):
2     # Kasus 1: list kosong
3     if self.head_chron is None:
4         self.head_chron = new_node
5         self.tail_chron = new_node
6     # Kasus 2: insert di depan (date paling kecil)
7     elif new_node.date < self.head_chron.date:
8         new_node.next_chron = self.head_chron
9         self.head_chron.prev_chron = new_node
10        self.head_chron = new_node
11    # Kasus 3: insert di belakang (date paling besar)
12    elif new_node.date > self.tail_chron.date:
13        new_node.prev_chron = self.tail_chron
14        self.tail_chron.next_chron = new_node
15        self.tail_chron = new_node
16    # Kasus 4: insert di tengah
17    else:
18        cur = self.head_chron
19        while cur is not None and cur.date < new_node.date:
20            cur = cur.next_chron
21        new_node.next_chron = cur
22        new_node.prev_chron = cur.prev_chron
23        cur.prev_chron.next_chron = new_node
24        cur.prev_chron = new_node
```

Kode 3 — Insert Alphabetical: Sorted Doubly Linked List (by Title)

Sama dengan insert chronological, namun key yang digunakan adalah title (string). Sorting dilakukan secara lexicographic (A → Z) mengacu pada konsep sorted doubly linked list Section 9.1.

```

3. Insert Alphabetical - Sorted Doubly Linked List

1 def _insert_alpha(self, new_node):
2     # Kasus 1: list kosong
3     if self.head_alpha is None:
4         self.head_alpha = new_node
5         self.tail_alpha = new_node
6     # Kasus 2: insert di depan (title paling awal abjad)
7     elif new_node.title < self.head_alpha.title:
8         new_node.next_alpha = self.head_alpha
9         self.head_alpha.prev_alpha = new_node
10        self.head_alpha = new_node
11    # Kasus 3: insert di belakang (title paling akhir abjad)
12    elif new_node.title > self.tail_alpha.title:
13        new_node.prev_alpha = self.tail_alpha
14        self.tail_alpha.next_alpha = new_node
15        self.tail_alpha = new_node
16    # Kasus 4: insert di tengah
17    else:
18        cur = self.head_alpha
19        while cur is not None and cur.title < new_node.title:
20            cur = cur.next_alpha
21        new_node.next_alpha = cur
22        new_node.prev_alpha = cur.prev_alpha
23        cur.prev_alpha.next_alpha = new_node
24        cur.prev_alpha = new_node

```

Kode 4 — Traversal: Chronological & Alphabetical View

Traversal maju dari head menggunakan pointer `next_chron` atau `next_alpha`, sesuai Listing 9.2 (revTraversal) yang dapat berjalan ke depan maupun ke belakang. Loop berhenti saat `curNode` bernilai `None`.

```

4. Traversal - Chronological & Alphabetical View

1 def view_chronological(self):
2     print('=== Chronological View ===')
3     cur = self.head_chron
4     while cur is not None:
5         print(f' [{cur.date}] {cur.title}')
6         cur = cur.next_chron
7
8 def view_alphabetical(self):
9     print('=== Alphabetical View ===')
10    cur = self.head_alpha
11    while cur is not None:
12        print(f' {cur.title}')
13        cur = cur.next_alpha

```

Kode 5 — Multi-Linked Tag Chains (Section 9.3)

Setiap tag memiliki head pointer sendiri dalam dictionary `tag_heads`. Ketika note baru ditambahkan, node-nya di-insert ke chain tag yang relevan. Ini mengacu pada Gambar 9.10 di mana satu node dapat menjadi bagian dari beberapa chain berbeda.

```
5. Multi-Linked Tag Chains (Section 9.3)

1 def _insert_tag_chains(self, new_node):
2     # Untuk setiap tag yang dimiliki note ini
3     for tag in new_node.tags:
4         if tag not in self.tag_heads:
5             # Tag baru: jadikan node ini sebagai head
6             self.tag_heads[tag] = new_node
7             new_node.next_by_tag[tag] = None
8         else:
9             # Tag sudah ada: sisipkan di depan chain
10            new_node.next_by_tag[tag] = self.tag_heads[tag]
11            self.tag_heads[tag] = new_node
12
13 def get_notes_by_tag(self, tag):
14     result = []
15     cur = self.tag_heads.get(tag, None)
16     while cur is not None:
17         result.append(cur.title)
18         cur = cur.next_by_tag.get(tag, None)
19     return result
```

Kode 6 — CircularSyncBuffer: Circular Linked List (Section 9.2)

Buffer melingkar dengan kapasitas tetap untuk menyimpan N perubahan terbaru. `listRef` menunjuk ke node terakhir ditambahkan. Saat penuh, node terlama (`listRef.next`) ditimpa — prinsip round-robin yang dijelaskan Section 9.2. Traversal mengikuti Listing 9.5.

```

6. SyncEntry Node & CircularSyncBuffer (Section 9.2)

1 class SyncEntry:
2     def __init__(self, message):
3         self.message = message
4         self.next = None
5
6 class CircularSyncBuffer:
7     def __init__(self, capacity=10):
8         self.capacity = capacity
9         self.listRef = None # pointer ke node terakhir (Section 9.2)
10        self.size = 0
11
12    def record_change(self, message):
13        new_node = SyncEntry(message)
14        if self.listRef is None:
15            # List kosong: node menunjuk ke dirinya sendiri (circular)
16            self.listRef = new_node
17            new_node.next = new_node
18            self.size = 1
19        elif self.size < self.capacity:
20            # Belum penuh: insert setelah listRef, jadikan listRef baru
21            new_node.next = self.listRef.next
22            self.listRef.next = new_node
23            self.listRef = new_node
24            self.size += 1
25        else:
26            # Buffer penuh: timpa node paling lama (listRef.next)
27            self.listRef.next.message = message
28            self.listRef = self.listRef.next
29
30    def get_recent_changes(self):
31        # Traversal circular sesuai Listing 9.5
32        if self.listRef is None:
33            return []
34        result = []
35        curNode = self.listRef
36        done = False
37        while not done:
38            curNode = curNode.next
39            result.append(curNode.message)
40            done = curNode is self.listRef
41        return result

```

Kode 7 — Kelas Utama NoteApp

NoteApp menyatukan ketiga struktur: dua sorted doubly linked list (chronological & alphabetical), dictionary tag chains (multi-linked), dan circular sync buffer. Method `add_note()` melakukan insert ke semua chain sekaligus dalam satu pemanggilan.


```

7. Kelas Utama NoteApp

1 class NoteApp:
2     def __init__(self):
3         # Doubly linked - chain chronological
4         self.head_chron = None
5         self.tail_chron = None
6         # Doubly linked - chain alphabetical
7         self.head_alpha = None
8         self.tail_alpha = None
9         # Multi-linked - tag head pointers
10        self.tag_heads = {}
11        # Circular buffer untuk sync tracking
12        self.sync_buffer = CircularSyncBuffer(capacity=10)
13
14    def add_note(self, title, content, date, tags):
15        new_node = NoteNode(title, content, date, tags)
16        # Insert ke semua chain sekaligus
17        self._insert_chron(new_node)
18        self._insert_alpha(new_node)
19        self._insert_tag_chains(new_node)
20        # Catat perubahan ke sync buffer
21        self.sync_buffer.record_change(f'ADDED: {title} [{date}]')
22        print(f'Note '{title}' berhasil ditambahkan.")
23
24    def view_sync_log(self):
25        print('=== Recent Sync Changes ===')
26        for entry in self.sync_buffer.get_recent_changes():
27            print(f' {entry}')

```

Kode 8 — Demo Program & Output

Contoh penggunaan lengkap: menambahkan 3 note dengan berbagai tag, menampilkan kedua view (chronological dan alphabetical), mencari note berdasarkan tag, dan melihat log sync.

```

8. Demo Program & Output

1 # ===== DEMO =====
2 app = NoteApp()
3 app.add_note('Belajar Python', 'Isi note...', '2025-01-10', ['python', 'belajar'])
4 app.add_note('AI Overview', 'Isi note...', '2025-01-08', ['ai', 'belajar'])
5 app.add_note('Python Tips', 'Isi note...', '2025-01-12', ['python'])
6
7 app.view_chronological()
8 app.view_alphabetical()
9 print('Notes tag python :', app.get_notes_by_tag('python'))
10 print('Notes tag belajar:', app.get_notes_by_tag('belajar'))
11 app.view_sync_log()
12
13 # OUTPUT:
14 # === Chronological View ===
15 # [2025-01-08] AI Overview
16 # [2025-01-10] Belajar Python
17 # [2025-01-12] Python Tips
18 #
19 # === Alphabetical View ===
20 # AI Overview
21 # Belajar Python
22 # Python Tips
23 #
24 # Notes tag python : ['Python Tips', 'Belajar Python']
25 # Notes tag belajar: ['AI Overview', 'Belajar Python']
26 #
27 # === Recent Sync Changes ===
28 # ADDED: Belajar Python [2025-01-10]
29 # ADDED: AI Overview [2025-01-08]
30 # ADDED: Python Tips [2025-01-12]

```